

YAML Configuration Examples for `hyper2kvm.py` (Cookbook, with vSphere Control/Data Plane)

This page is a **copy-paste cookbook** for running `hyper2kvm.py` using YAML configs.

It covers the big three you already run in production (**local**, **live-fix**, **fetch-and-fix**) and adds the “missing” ones that show up in real migrations (**ova**, **ovf**, **daemon**, **vsphere/pyvmomi**). It also captures the **design intent** behind the knobs: `hyper2kvm` splits vSphere into **control-plane** (inventory/orchestration) and **data-plane** (moving bytes), because mixing them is how tools become slow and haunted.

Tip: keep one `base.yaml` with defaults, and override per-customer / per-VM in a tiny overlay file.

Prerequisites

Before following this guide, you should have:

- ✓ Completed the [Installation](#)
- ✓ Familiarity with basic hyper2kvm concepts
- ✓ Root/sudo access to your system
- ✓ Source VM files ready for migration

Table of contents

- [Running configs](#)
- [Mental model: modes + planes](#)
- [Common keys](#)
- [1. Local mode](#)
- [2. Live-fix mode](#)

- [3. Fetch-and-fix mode](#)
 - [4. OVA mode](#)
 - [5. OVF mode](#)
 - [6. Daemon mode](#)
 - [7. vSphere / pyvmomi mode](#)
 - [8. Batch migration mode](#)
 - [9. Migration profiles](#)
 - [10. Pre/Post hooks](#)
 - [11. Libvirt XML import](#)
 - [12. Network and storage mapping](#)
 - [Base + overrides pattern](#)
 - [Troubleshooting patterns](#)
-

Running configs

Run a config by selecting a command (mode):

```
sudo ./hyper2kvm.py --config example.yaml local
```

Merge multiple configs (later overrides earlier):

```
sudo ./hyper2kvm.py --config base.yaml --config overrides.yaml local
'''yaml

### Multi-VM configs

If you use a `vms:` list, the tool treats top-level keys as defaults and
allows per-VM overrides:

'''yaml
vms:
  - vmdk: /path/to/vm1.vmdk
    to_output: vm1.qcow2
  - vmdk: /path/to/vm2.vmdk
    to_output: vm2.qcow2
    compress: false # override only for vm2

compress: true # default for all VMs
out_format: qcow2
```

```

'''yaml

---

## Mental model: modes + planes

### Modes (what you're doing)

* **local**: convert/fix from local VMDK/OVF/OVA inputs
* **live-fix**: fix an already-booted guest via SSH
* **fetch-and-fix**: pull from ESXi datastore via SSH, then fix offline
* **vsphere**: talk to vCenter/ESXi APIs (pyvmomi): discover/download/snapshot/
  CBT
* **daemon**: watch a directory and auto-convert

### Planes (how bytes move)

* **Control-plane (pyvmomi / pyVim / pyVmomi)**: inventory, datacenter/host
  resolution, snapshots, CBT maps, datastore browsing (listing).
* **Data-plane**:

  * **HTTP `/folder`**: byte-for-byte download of datastore files using vCenter
    session cookie
  * **VDDK client**: single-disk raw pull through VDDK (when available)
  * **Internal converters**: OVA/VHD/AMI/VMDK extraction and conversion to
    qcow2/raw

**Rule of thumb**: use the **least invasive** data-plane that solves your goal:

* Need qcow2 + conversion? → hyper2kvm internal converters
* Need raw datastore bytes? → HTTP `/folder` download-only
* Need one disk fast via ESXi? → VDDK pull
* Need incremental sync? → CBT + HTTP Range reads

---

## Common keys

These keys show up across multiple modes.

### Logging / diagnostics

* `verbose`: `0|1|2` (or `-v/-vv`)
* `log_file`: write logs to a file
* `report`: write a Markdown report (recommended for long conversions)
* `checksum`: compute SHA256 of final output

### Safety

* `dry_run`: preview changes (does not modify the guest image / output)

```

```

* `no_backup`: skip backups inside guest (**dangerous**)
* `enable_recovery`: checkpoints for long ops (recommended)
* `parallel_processing`: parallelize batch conversion

### Conversion

* `flatten`: flatten snapshot chain first (recommended if snapshots exist)
* `flatten_format`: `qcow2` or `raw`
* `to_output`: final output file name/path
* `out_format`: `qcow2|raw|vdi`
* `compress`: qcow2 compression
* `compress_level`: `1..9`
* `resize` : `+10G` (grow by) or `50G` (set total)

### Fixes (offline or live, depending on mode)

* `fstab_mode`: `stabilize-all` (recommended) | `bypass-only` | `noop`
* `print_fstab`: print fstab before/after
* `no_grub`: skip grub root= changes and device.map cleanup
* `regen_initramfs`: regenerate initramfs + grub config (best-effort)
* `remove_vmware_tools`: remove VMware tools (Linux guests)
* `cloud_init_config`: inject cloud-init config (Linux guests)

### Tests (optional)

* `libvirt_test`: define + boot the VM and verify it reaches RUNNING
* `qemu_test`: basic qemu launch smoke
* `uefi`: use UEFI (OVMF) for test VM
* `headless`: no graphics device (important for headless servers)

---

## 1. Local mode (offline conversion from local VMDK)

Use when you already have VMDK(s) locally (descriptor/monolithic, chain, etc.).

### Basic single VM

```yaml
command: local
vmdk: /path/to/vm.vmdk
output_dir: ./out

flatten: true
to_output: vm-fixed.qcow2
out_format: qcow2
compress: true
compress_level: 6

```

```

fstab_mode: stabilize-all
regen_initramfs: true
remove_vmware_tools: true

report: local-report.md
verbose: 1
```yaml

### Advanced: Windows virtio injection + safety backups

```yaml
command: local
vmdk: /path/to/windows-vm.vmdk
output_dir: ./out

to_output: win-fixed.qcow2
out_format: qcow2
compress: true

virtio_drivers_dir: /path/to/virtio-win
(If you wired these knobs) keep BCD backups + registry safety:
windows_bcd_backup: true
windows_reg_backup: true

enable_recovery: true
report: windows-report.md
checksum: true
verbose: 2
```yaml

### Multi-VM batch (shared defaults + overrides)

```yaml
output_dir: ./out
workdir: ./out/work

flatten: true
out_format: qcow2
compress: true
compress_level: 6

fstab_mode: stabilize-all
regen_initramfs: true
enable_recovery: true
parallel_processing: true
verbose: 1

```

```

vms:
 - vmdk: /path/to/vm1.vmdk
 to_output: vm1.qcow2
 resize: +10G
 - vmdk: /path/to/vm2.vmdk
 to_output: vm2.qcow2
 remove_vmware_tools: false
 - vmdk: /path/to/win.vmdk
 to_output: win.qcow2
 virtio_drivers_dir: /path/to/virtio-win
 """yaml

2. Live-fix mode (live fixes on running VM via SSH)

Use when the guest is already booted (post-
migration fixes without touching disk images).

Basic live-fix

 """yaml
command: live-fix
host: 192.168.1.100
user: root
port: 22
sudo: true

fstab_mode: stabilize-all
print_fstab: true
regen_initramfs: true
remove_vmware_tools: true

verbose: 2
log_file: live-fix.log
 """yaml

Advanced: custom key + SSH opts + dry-run

 """yaml
command: live-fix
host: vm.example.com
user: admin
identity: ~/.ssh/custom_key

ssh_opt:
 - "-o StrictHostKeyChecking=no"
 - "-o ConnectTimeout=30"

```

```

fstab_mode: bypath-only
no_grub: true
dry_run: true
verbose: 2
```yaml
---

## 3. Fetch-and-fix mode (fetch from ESXi and fix offline)

Use when you can SSH to ESXi and pull VMDKs from datastore paths.

### Basic fetch-and-fix

```yaml
command: fetch-and-fix
host: esxi.example.com
user: root
port: 22

remote: /vmfs/volumes/datastore1/vm/vm.vmdk
fetch_dir: ./downloads

flatten: true
to_output: esxi-vm-fixed.qcow2
out_format: qcow2
compress: true
report: fetch-fix.md
```yaml

### Advanced: full chain + test boot

```yaml
command: fetch-and-fix
host: esxi-host
identity: ~/.ssh/esxi_key
remote: /vmfs/volumes/ds1/snapvm/snapvm.vmdk

fetch_all: true
flatten: true
resize: 50G

fstab_mode: stabilize-all
regen_initramfs: true

libvirt_test: true
vm_name: esxi-test-vm
uefi: true

```

```

headless: true
timeout: 120

enable_recovery: true
report: esxi-report.md
'''yaml

4. OVA mode (extract from OVA and convert)

OVA is a tarball containing OVF + disk(s).

Basic OVA extract + convert

'''yaml
command: ova
ova: /path/to/appliance.ova
output_dir: ./out

flatten: true
to_output: appliance.qcow2
out_format: qcow2
compress: true
compress_level: 6

report: ova-report.md
'''yaml

5. OVF mode (parse OVF and convert)

Use when you have an `*.ovf` descriptor and disks alongside it.

Basic OVF parse + convert

'''yaml
command: ovf
ovf: /path/to/vm.ovf
output_dir: ./out

flatten: true
to_output: ovf-vm.qcow2
out_format: qcow2
compress: true
'''yaml

```



```
OVF "fragile guest" mode (avoid GRUB mutation)
```

```
```yaml
command: ovf
ovf: /path/to/vm.ovf
output_dir: ./out

fstab_mode: stabilize-all
no_grub: true
regen_initramfs: false

flatten: true
to_output: ovf-safe.qcow2
out_format: qcow2
report: ovf-safe.md
```yaml
```

```

```

```
6. Daemon mode (watch a directory and auto-convert)
```

Use for pipelines: "drop VMDKs here → get qcow2 there".

```
Basic daemon watch
```

Daemon mode uses **\*\*watchdog\*\*** to monitor a directory for new disk files and automatically processes them through the conversion pipeline.

**\*\*Supported file types:\*\***

- ``.vmdk`` - VMware disks
- ``.ova``, ``.ovf`` - OVF archives
- ``.vhd``, ``.vhdx`` - Hyper-V disks
- ``.raw``, ``.img`` - Raw disk images
- ``.ami`` - AWS AMI images

**\*\*How it works:\*\***

1. Monitors ``.watch_dir`` **for** new files using filesystem events
2. When a file appears, queues it for processing
3. Processes each file through the full conversion pipeline
4. Archives completed files to ``.processed/`` subdirectory
5. Moves failed files to ``.errors/`` subdirectory
6. Runs continuously until stopped (**Ctrl+C** or **SIGTERM**)

```
```yaml
command: daemon
daemon: true

# Directory to watch (will be created if missing)
watch_dir: /srv/incoming-vmdk
```

```
# Output directory for converted VMs
output_dir: /srv/out

# Working directory for temporary files
workdir: /srv/out/work

# Conversion options
flatten: true
out_format: qcow2
compress: true
compress_level: 6

# Enable recovery mode for resumable conversions
enable_recovery: true

# Logging
log_file: /var/log/hyper2kvm-daemon.log
verbose: 1

# Guest OS fixes (applied to all conversions)
fstab_mode: stabilize-all
regen_initramfs: true
```

7. vSphere / pyvmomi mode (discovery, download, CBT)

This mode talks to vCenter/ESXi APIs using **pyvmomi**.

Design intent (why these actions exist)

- `list_vm_names` , `get_vm_by_name` , `vm_disks` , `select_disk` are **control-plane**: fast inventory and inspection.
- `download_datastore_file` , `download_vm_disk` , `download_only_vm` are **data-plane via HTTP /folder** : pull raw bytes using the session cookie.
- `create_snapshot` , `enable_cbt` , `query_changed_disk_areas` are **control-plane** orchestration.
- `cbt_sync` is **hybrid**: control-plane computes ranges, data-plane applies ranged reads.

Connection block (common)

```
command: vsphere
vcenter: vcenter.example.com
vc_user: administrator@vsphere.local
vc_password_env: VC_PASSWORD
vc_insecure: true # set false in real environments with trusted certs
vc_port: 443
json: true
```

List VM names (bulk)

```
command: vsphere
vcenter: vcenter.example.com
vc_user: administrator@vsphere.local
vc_password_env: VC_PASSWORD
vc_insecure: true
```

```
vs_action: list_vm_names
json: true
```yaml
```

### Get VM details

```
```yaml
command: vsphere
vcenter: vcenter.example.com
vc_user: administrator@vsphere.local
vc_password_env: VC_PASSWORD
vc_insecure: true
```

```
vs_action: get_vm_by_name
name: myVM
json: true
```yaml
```

### List disks for a VM

```
```yaml
command: vsphere
vcenter: vcenter.example.com
vc_user: administrator@vsphere.local
vc_password_env: VC_PASSWORD
vc_insecure: true
```

```

vs_action: vm_disks
vm_name: myVM
json: true
```yaml

Download a datastore file (HTTP `/folder`)

```yaml
command: vsphere
vcenter: vcenter.example.com
vc_user: administrator@vsphere.local
vc_password_env: VC_PASSWORD
vc_insecure: true

dc_name: ha-datacenter
vs_action: download_datastore_file
datastore: datastore1
ds_path: "myVM/myVM.vmdk"          # IMPORTANT: datastore-relative (your CLI
                                   builds URL quoting)
local_path: ./downloads/myVM.vmdk
chunk_size: 1048576
json: true
```yaml

Download a VM disk (select by index/label → backing filename → HTTP pull)

```yaml
command: vsphere
vcenter: vcenter.example.com
vc_user: administrator@vsphere.local
vc_password_env: VC_PASSWORD
vc_insecure: true

vs_action: download_vm_disk
vm_name: myVM
disk: 0
local_path: ./downloads/myVM-disk0.vmdk
chunk_size: 1048576
json: true
```yaml

Download-only VM folder pull (byte-for-byte VM directory)

```yaml
command: vsphere
vcenter: vcenter.example.com
vc_user: administrator@vsphere.local
vc_password_env: VC_PASSWORD
vc_insecure: true

```

```

vs_action: download_only_vm
vm_name: myVM
output_dir: ./downloads/myVM-folder

vs_include_glob:
  - "*.vmx"
  - "*.vmdk"
  - "*.nvram"
  - "*.vmsd"
  - "*.vmxf"
vs_exclude_glob:
  - "*.log"
  - "*.lck"
  - "*.vswp"
  - "*.vmem"

vs_concurrency: 6
vs_max_files: 5000
vs_fail_on_missing: false

dc_name: ha-datacenter
json: true
```yaml

Snapshot + CBT + changed areas (control-plane)

```yaml
command: vsphere
vcenter: vcenter.example.com
vc_user: administrator@vsphere.local
vc_password_env: VC_PASSWORD
vc_insecure: true

# snapshot
vs_action: create_snapshot
vm_name: myVM
name: hyper2kvm-pre-migration
quiesce: true
memory: false
description: "Created by hyper2kvm"
json: true
```yaml

```yaml
command: vsphere
vcenter: vcenter.example.com
vc_user: administrator@vsphere.local

```

```

vc_password_env: VC_PASSWORD
vc_insecure: true

# enable CBT
vs_action: enable_cbt
vm_name: myVM
json: true
```yaml

```yaml
command: vsphere
vcenter: vcenter.example.com
vc_user: administrator@vsphere.local
vc_password_env: VC_PASSWORD
vc_insecure: true

# query CBT ranges
vs_action: query_changed_disk_areas
vm_name: myVM
snapshot_name: hyper2kvm-cbt
disk: 0
start_offset: 0
change_id: "*"
json: true
```yaml

CBT delta sync (base download once, then patch deltas)

```yaml
command: vsphere
vcenter: vcenter.example.com
vc_user: administrator@vsphere.local
vc_password_env: VC_PASSWORD
vc_insecure: true

vs_action: cbt_sync
vm_name: myVM
disk: 0
local_path: ./downloads/myVM-disk0.vmdk

enable_cbt: true
snapshot_name: hyper2kvm-cbt
change_id: "*"

dc_name: ha-datacenter
json: true
```yaml

```

```

Base + overrides pattern

`base.yaml` (team defaults)

```yaml
output_dir: ./out
workdir: ./out/work

enable_recovery: true
fstab_mode: stabilize-all
print_fstab: true
regen_initramfs: true

flatten: true
out_format: qcow2
compress: true
compress_level: 6

checksum: true
report: report.md
verbose: 1
```yaml

`overrides.yaml` (per VM / per customer)

```yaml
vmdk: /path/to/customer/vm.vmdk
to_output: customer-vm.qcow2

remove_vmware_tools: false
no_grub: true      # fragile guest
dry_run: false
```yaml

Run:

```bash
sudo ./hyper2kvm.py --config base.yaml --config overrides.yaml local
```yaml

Troubleshooting patterns

"Headless server" libvirt test fails with SDL / XDG errors

```

```

Use:

```yaml
libvirt_test: true
headless: true
```yaml

Windows boots to recovery / INACCESSIBLE_BOOT_DEVICE after migration

Usually storage driver boot-start + CriticalDeviceDatabase.
Ensure virtio injection is enabled and you keep safety backups:

```yaml
virtio_drivers_dir: /path/to/virtio-win
# windows_bcd_backup: true
# windows_reg_backup: true
```yaml

Snapshot chain conversions are slow / fail

* `flatten: true`
* set `workdir` on fast SSD
* `enable_recovery: true`
* avoid extreme `compress_level` on CPU-bound hosts

vSphere downloads fail with 404/permission issues

Common causes:

* wrong `dc_name` (`dcPath=` matters)
* `ds_path` not datastore-relative or not properly quoted
* vCenter session expired (cookie) → reconnect
* TLS verification mismatch (`vc_insecure`)

"Required arg missing" even though it's in YAML

Usually key mismatch vs argparse destination name.
Prefer the exact names your CLI expects (`to_output`, `out_format`, `vm_name`,
`vs_action`, etc.).

8. Batch migration mode (multi-VM parallel conversions)

Use case: Convert multiple VMs in parallel with centralized orchestration

Batch manifest (`batch-migration.json`):
```json
{

```



```

"batch_version": "1.0",
"batch_metadata": {
  "batch_id": "datacenter-migration-2026",
  "parallel_limit": 4,
  "continue_on_error": true
},
"shared_config": {
  "output_directory": "/mnt/converted-vms",
  "profile": "production"
},
"vms": [
  {
    "manifest": "/manifests/web-server-01.json",
    "priority": 0,
    "overrides": {
      "output": {
        "vm_name": "web-server-01-kvm"
      }
    }
  },
  {
    "manifest": "/manifests/database-01.json",
    "priority": 1
  },
  {
    "manifest": "/manifests/app-server-01.json",
    "priority": 2
  }
]
}

```

Run batch conversion:

```
sudo hyper2kvm --batch-manifest batch-migration.json --batch-parallel 4
```

With checkpoint/resume (crash recovery):

```

# First run (may crash/fail midway)
sudo hyper2kvm --batch-manifest batch-migration.json --batch-parallel 4

# Resume from checkpoint after crash
sudo hyper2kvm --batch-manifest batch-migration.json --batch-resume

```

Key features: - Parallel execution (configurable worker limit) - Priority-based VM ordering
- Per-VM error isolation with `continue_on_error` - Aggregate progress reporting -
Checkpoint/resume for crash recovery - Real-time progress persistence to JSON

9. Migration profiles (reusable configuration templates)

Use case: Standardize conversion settings across environments

Individual VM manifest with profile (`vm-manifest.json`):

```
{
  "manifest_version": "1.0",
  "profile": "production",
  "profile_overrides": {
    "pipeline": {
      "convert": {
        "compress_level": 9
      }
    }
  },
  "source": {
    "type": "local",
    "path": "/vmdk/rhel9-server.vmdk"
  },
  "output": {
    "vm_name": "rhel9-production"
  }
}
```

Custom profile (`custom-profile.yaml`):

```
# Extends built-in production profile
extends: production

pipeline:
  fix:
    enabled: true
    backup: true
    # grub is auto-handled
    regen_initramfs: true
    inject_drivers:
      - virtio_net
      - virtio_blk
```

```

    - virtio_scsi
  convert:
    enabled: true
    compress: true
    compress_level: 6
  validate:
    enabled: true

  output:
    format: "qcow2"

  libvirt_integration:
    enabled: true
    define_domain: true
    import_to_pool: "production-vms"
    create_snapshot: true
    snapshot_name: "pre-first-boot"

```

Built-in profiles: - `production` - Full conversion with validation and compression - `testing` - Fast conversion, skip validation - `minimal` - Extract and fix only, no conversion - `fast` - Minimal fixes, no compression - `windows` - Windows-specific optimizations - `archive` - Maximum compression for archival - `debug` - Verbose logging, keep all intermediates

Run with custom profile:

```

sudo hyper2kvm --config vm-manifest.json \
  --custom-profile-dir /path/to/profiles local

```

10. Pre/Post hooks (automation and integration)

Use case: Integrate custom scripts, validation, and notifications into the conversion pipeline

Manifest with hooks (`vm-with-hooks.json`):

```

{
  "manifest_version": "1.0",
  "source": {
    "type": "local",
    "path": "/vmdk/rhel9.vmdk"
  }
}

```

```

    },
    "hooks": {
        "pre_extraction": [
            {
                "type": "script",
                "path": "/scripts/backup-source.sh",
                "env": {
                    "SOURCE_PATH": "{{ source_path }}",
                    "VM_NAME": "{{ vm_name }}"
                },
                "timeout": 600,
                "continue_on_error": false
            }
        ],
        "post_fix": [
            {
                "type": "python",
                "module": "custom_validators",
                "function": "check_boot_config",
                "args": {
                    "disk_path": "{{ extracted_disk_path }}"
                },
                "timeout": 300
            }
        ],
        "post_convert": [
            {
                "type": "http",
                "url": "https://monitoring.example.com/webhook",
                "method": "POST",
                "headers": {
                    "Content-Type": "application/json"
                },
                "body": {
                    "vm_name": "{{ vm_name }}",
                    "status": "converted",
                    "output_path": "{{ output_path }}"
                },
                "timeout": 30,
                "continue_on_error": true
            }
        ]
    }
}

```

With retry logic (exponential backoff):

```
{
  "hooks": {
    "post_convert": [
      {
        "type": "http",
        "url": "https://api.example.com/notify",
        "method": "POST",
        "body": {"vm": "{{ vm_name }}"},
        "retry": {
          "max_retries": 3,
          "retry_delay": 5,
          "retry_strategy": "exponential",
          "max_delay": 60
        },
        "continue_on_error": true
      }
    ]
  }
}
```

Available hook stages: - `pre_extraction` - Before disk extraction -
`post_extraction` - After extraction, before fixes - `pre_fix` - Before offline fixes -
`post_fix` - After fixes, before conversion - `pre_convert` - Before format conversion -
`post_convert` - After conversion, before validation - `post_validate` - After
validation complete

Template variables (15+ available): - `{{ vm_name }}`, `{{ source_path }}`,
`{{ output_path }}` - `{{ output_format }}`, `{{ extracted_disk_path }}` -
`{{ work_dir }}`, `{{ timestamp }}`, `{{ batch_id }}`

11. Libvirt XML import (convert existing VMs)

Use case: Import and convert VMs defined in libvirt XML format

Libvirt XML import:

```
# Parse libvirt domain XML and convert
sudo hyper2kvm libvirt-xml \
  --xml-path /etc/libvirt/qemu/myvm.xml \
  --output-dir /converted
```

With automatic libvirt integration:

```
{
  "manifest_version": "1.0",
  "source": {
    "type": "libvirt-xml",
    "xml_path": "/etc/libvirt/qemu/myvm.xml"
  },
  "libvirt_integration": {
    "enabled": true,
    "define_domain": true,
    "import_to_pool": "converted-vms",
    "auto_start": false,
    "autostart_on_boot": false,
    "create_snapshot": true,
    "snapshot_name": "pre-first-boot"
  },
  "output": {
    "vm_name": "myvm-converted",
    "format": "qcow2"
  }
}
```

Features: - Parses disk paths and formats from XML - Detects firmware type (BIOS/UEFI)
- Extracts network configuration - Reads memory/CPU settings - Optional SHA256 checksum computation - Auto-imports to libvirt storage pools - Creates pre-boot snapshots

12. Network and storage mapping (infrastructure transformation)

Use case: Transform source network/storage configurations to target infrastructure

Network mapping (`vm-with-mapping.yaml`):

```
manifest_version: "1.0"

source:
  type: local
  path: /vmdk/rhel9.vmdk

# Network mapping configuration
```

```

network_mapping:
  source_networks:
    "VM Network": "br0"
    "DMZ Network": "br-dmz"
    "Internal": "br-internal"
  mac_address_policy: preserve # preserve|regenerate|custom
  mac_address_overrides:
    "00:50:56:ab:cd:ef": "52:54:00:12:34:56"

# Storage mapping configuration
storage_mapping:
  default_pool: "production-vms"
  disk_mappings:
    boot: "/mnt/nvme/boot-disks"
    data: "/mnt/storage/data-disks"
  format_override: "qcow2"

output:
  vm_name: rhel9-mapped

```

Batch with shared mappings:

```

{
  "batch_version": "1.0",
  "shared_config": {
    "network_mapping": {
      "source_networks": {
        "VM Network": "br0"
      },
      "mac_address_policy": "preserve"
    },
    "storage_mapping": {
      "default_pool": "vms",
      "format_override": "qcow2"
    }
  },
  "vms": [
    {"manifest": "/manifests/vm1.json"},
    {"manifest": "/manifests/vm2.json"}
  ]
}

```

Features: - Source network to target bridge mapping - MAC address policies (preserve/regenerate/custom) - Per-disk storage mappings - Format override per disk or globally - Applied during libvirt XML generation

Next Steps

Continue your migration journey:

- [CLI Reference](#) - Complete command options
- [YAML Examples](#) - Configuration templates
- [Cookbook](#) - Common scenarios
- [Troubleshooting](#) - When things go wrong

Getting Help

Found an issue? [Report it on GitHub](#)